

ANÁLISIS DEL RETO

Student-1, Yury Dayana Velasquez Alvarez, y.velasquez@uniandes.edu.co

Student-2, Nelson Fabian Reyes Marquez, 202515993, n.reyesm@uniandes.edu.co

Student-3, Luis Esteban Acevedo Ladino, 202521217, le.acevedo@uniandes.edu.co

Requerimiento 1

Descripción

Para la implementación del requerimiento se realizó un recorrido secuencial sobre la estructura **array_list** que almacena los computadores en el catálogo.

Durante el recorrido se filtraron únicamente aquellos computadores cuya marca coincidiera con la marca ingresada por el usuario (normalizando el texto mediante `strip()` y `lower()` para evitar inconsistencias por mayúsculas o espacios).

En una **única pasada** sobre los datos se calcularon:

- Contador total de computadores de la marca
- Sumas acumuladas para el cálculo de promedios
- Valores mínimos y máximos para precio, RAM, VRAM, número de núcleos y año de lanzamiento
- Modelos correspondientes al computador de mayor y menor precio, considerando desempate por menor peso

De esta manera se evita realizar múltiples recorridos sobre la estructura o emplear ciclos anidados, optimizando la complejidad temporal del algoritmo.

Entrada	catalog: estructura que contiene la lista de computadores almacenada como arraylist. brand: nombre de la marca a consultar (string).
Salidas	El algoritmo retorna: • Tiempo de ejecución en milisegundos. • Número total de computadoras de la marca. • Promedio, mínimo y máximo de: ○ Precio ○ Memoria RAM ○ Memoria VRAM ○ Número de núcleos ○ Año de lanzamiento • Modelo del computador de mayor precio (desempate por menor peso). • Modelo del computador de menor precio (desempate por menor peso). • Diccionario consolidado con la información general calculada.
Implementado (Sí/No)	Si. Implementado por: Student-1

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tamaño del catálogo	$O(1)$
Normalización del texto de entrada	$O(1)$
Inicialización de variables acumuladoras	$O(1)$
Recorrido completo del catálogo	$O(n)$
Cálculo de promedios	$O(1)$
Construcción del diccionario de respuesta	$O(1)$
Cálculo del tiempo de ejecución	$O(1)$
TOTAL	$O(n)$

Análisis

La operación dominante del algoritmo es el recorrido completo del catálogo de computadores para filtrar por una marca específica.

Sea n el número total de computadores en el catálogo:

- El algoritmo realiza exactamente un recorrido completo de la estructura `array_list`.
- Dentro del ciclo, todas las funciones primitivas tales como comparaciones, sumas, asignaciones y verificaciones de mínimo/máximo son, de acuerdo a la teoría, de tiempo constante $O(1)$.
- No se emplean estructuras auxiliares proporcionales al tamaño de entrada.
- No se realizan recorridos adicionales o existen ciclos anidados.

Por tanto, la complejidad temporal de este requerimiento es:

$$O(n)$$

Esta complejidad es óptima, ya que para determinar la cantidad de computadores que pertenecen a una marca específica es necesario inspeccionar cada elemento al menos una vez.

En cuanto a la estructura de datos utilizada, se tomó la decisión de utilizar la estructura **`array_list`** en lugar de **`single_linked_list`**. Aunque ambas estructuras cuentan con una complejidad temporal de $O(n)$ para recorridos completos, se seleccionó **`array_list`** ya que ofrece mejor rendimiento práctico debido a su almacenamiento contiguo en memoria acceso $O(1)$ por índice. Por el contrario, **`single_linked_list`** resulta más eficiente al momento de hacer múltiples inserciones o eliminaciones dinámicas en posiciones intermedias, lo cual no es el caso en este requerimiento.

Finalmente, en las pruebas realizadas, el tiempo de ejecución crece de manera proporcional al tamaño del dataset, lo cual confirma el comportamiento lineal esperado del algoritmo.

Requerimiento 2

Descripción

Para la implementación del requerimiento se realizó un recorrido secuencial sobre la estructura arraylist que almacena los computadores en el catálogo.

Durante el recorrido se filtraron únicamente aquellos computadores cuyo precio estuviera dentro del rango [min_price, max_price] ingresado por el usuario.

En una única pasada sobre los datos se calcularon:

- Lista de computadoras dentro del rango de precios.
- Contador total de computadores que cumplen la condición.
- Sumas acumuladas para el cálculo de promedios de:
 - RAM
 - VRAM
 - Precio
- Computador más moderno (mayor año de lanzamiento; en caso de empate, mayor precio).
- Computador de menor precio dentro del rango.
- Computador de mayor precio dentro del rango.

De esta manera se evita realizar múltiples recorridos sobre la estructura o emplear ciclos anidados, optimizando la complejidad temporal del algoritmo.

Entrada	catalog: estructura que contiene la lista de computadores almacenada como arraylist. min_price: precio mínimo del rango (float). max_price: precio máximo del rango (float).
Salidas	El algoritmo retorna: • Lista de computadoras dentro del rango de precios.

	• Número total de computadores encontrados. • Promedio de: ○ Memoria RAM ○ Memoria VRAM ○ Precio • Computador más moderno (con desempate por mayor precio). • Computador de menor precio. • Computador de mayor precio. • Tiempo de ejecución en milisegundos.
Implementado (Sí/No)	Si. Implementado por: Student-2

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tamaño del catálogo	$O(1)$
creación de lista auxiliar	$O(1)$
Inicialización de variables	$O(1)$
Recorrido completo del catálogo	$O(n)$
Cálculo de promedios	$O(1)$
Cálculo del tiempo de ejecución	$O(1)$
Retornar resultados	$O(1)$
TOTAL	$O(n)$

Análisis

Sea n el número total de computadores en el catálogo.

La operación dominante del algoritmo es el recorrido completo del arraylist, el cual se ejecuta exactamente una vez mediante un ciclo for.

Dentro del ciclo se realizan únicamente operaciones de tiempo constante:

- Comparaciones numéricas.

- Conversión de tipos (float, int).
- Sumas acumuladas.
- Asignaciones.
- Verificaciones de mínimo, máximo y desempates.
- Inserción al final de la lista `add_last`, cuya complejidad es $O(1)$.

No se emplean ciclos anidados ni recorridos adicionales sobre la estructura principal. El cálculo de promedios se realiza fuera del ciclo en tiempo constante. Por lo tanto, la complejidad temporal total del algoritmo es:

$$O(n)$$

Complejidad espacial

Se utiliza una lista auxiliar para almacenar los computadores que cumplen el rango de precios. En el peor caso, todos los elementos del catálogo podrían cumplir la condición, por lo que el espacio adicional sería proporcional a n .

Por tanto, la complejidad espacial es:

$$O(n)$$

Conclusión

El algoritmo presenta complejidad temporal lineal $O(n)$, la cual es óptima, ya que para determinar qué computadores se encuentran dentro de un rango de precios específico es necesario inspeccionar cada elemento al menos una vez.

Las pruebas realizadas muestran que el tiempo de ejecución crece de manera proporcional al tamaño del dataset, confirmando el comportamiento lineal esperado.

Requerimiento 3

Descripción

Objetivo: Analizar computadores por marca y tier de CPU.

Procedimiento:

1. Recorre secuencialmente la lista de computadores (Array List)
2. Filtra por marca de CPU y CPU tier (comparación case-insensitive)
3. Para cada coincidencia:
 - Agrega el computador a una Single Linked List
 - Acumula: precio, RAM, VRAM, hilos de CPU
 - Actualiza diccionarios de conteo para GPU y año de lanzamiento
4. Determina GPU y año más frecuentes recorriendo los diccionarios (desempate alfabético)
5. Calcula promedios dividiendo acumulados entre el total filtrado
6. Retorna estadísticas y tiempo de ejecución

Complejidad: $O(n + m + k)$ donde n = total computadores, m = GPUs únicas, k = años únicos

Entrada	catalog: estructura que contiene la lista de computadores almacenada como arraylist. cpu_brand: cadena de caracteres (string) que indica la marca del procesador a filtrar.. cpu_tier: cadena de caracteres (string) que indica la gama del procesador a filtrar.
Salidas	El algoritmo retorna: - Total de computadores que cumplieron el filtro de marca y gama de CPU. - Precio promedio. - RAM promedio. - VRAM promedio. - Número de hilos (threads) de CPU promedio. - Marca de GPU (tarjeta gráfica) más común y su número de apariciones. - Año de lanzamiento más común y su número de apariciones. - Tiempo de ejecución en milisegundos.

Implementado (Sí/No)	Implementado por student 3
----------------------	----------------------------

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tiempo de inicio	$O(1)$
Normalizar textos de los filtros (lower)	$O(1)$
Obtener tamaño del catálogo	$O(1)$
Inicializar variables, listas auxiliares y diccionarios vacíos	$O(1)$
Recorrido completo del catálogo principal (filtro por <code>cpu_brand</code> y <code>cpu_tier</code>)	$O(N)$
Inserción en la lista enlazada auxiliar (por elemento que cumple el filtro)	$O(1)$
Acumulación de sumas (precio, ram, vram, hilos)	$O(1)$
Actualización de frecuencias en diccionarios (GPU y año de lanzamiento)	$O(1)$
Recorrido de diccionario para buscar la marca de GPU más común	$O(G)$
Recorrido de diccionario para buscar el año de lanzamiento más común	$O(Y)$
Cálculo de promedios	$O(1)$
Cálculo del tiempo de ejecución	$O(1)$
TOTAL	$O(N)$

Análisis

En este requerimiento la operación que más complejidad temporal implica, es el recorrido por todo el dataset en busca de items que pasen el filtro. Sea n el número total de computadores en el catálogo, m la cantidad de computadores que cumplen con el filtro (donde $m \leq n$), g la cantidad de marcas de GPU únicas (donde $g \leq m$) y y la cantidad de años únicos encontrados (donde $y \leq m$):

- El algoritmo realiza un recorrido único por el array list (catálogo) con tamaño N , para verificar si el elemento actual cumple o no con el filtro.
- En este ciclo, los elementos que cumplen con este filtro se añaden a una estructura auxiliar single linked list para consolidar los elementos filtrados.
- Simultáneamente (dentro del mismo ciclo), se acumulan los valores de las variables numéricas y se actualizan dos diccionarios nativos que llevan el conteo de apariciones de las GPU y los años de lanzamiento. Las operaciones de búsqueda, validación e incremento en los diccionarios (tablas hash) toman un tiempo promedio constante $O(1)$.
- Posteriormente, al finalizar el recorrido principal, se llama a la función auxiliar `get_most_common` dos veces. Estas llamadas realizan recorridos secuenciales cortos sobre las claves de los diccionarios (tamaño g para las GPUs y tamaño y para los años) para encontrar la llave con el valor máximo.

- No existen ciclos anidados dentro del procesamiento de datos principal.

Por lo tanto, se puede concluir que la complejidad temporal del requerimiento es la suma de los recorridos secuenciales realizados: $O(n) + O(g) + O(y)$. Para notación Big-O, la expresión para la complejidad temporal final se simplifica y clasifica formalmente como $O(n)$. Experimentalmente pudimos corroborar esta conclusión como se observa en el gráfico 1.

En cuanto a las Estructuras de Datos (TAD) usadas, se instancia una `single_linked_list` para agrupar dinámicamente el subconjunto de computadores filtrados. Aunque los elementos de esta lista no se recorren de nuevo (ya que los promedios se calculan con los acumuladores), el uso de esta estructura es ideal para almacenar una cantidad desconocida a priori de elementos, con un costo eficiente de $O(1)$ por inserción al final. Adicionalmente, el uso de diccionarios (hash maps) para llevar el conteo de frecuencias de GPUs y años demostró ser una decisión óptima y directa, ya que reemplaza la necesidad de hacer búsquedas secuenciales costosas, garantizando inserciones y actualizaciones en tiempo constante $O(1)$.

Requerimiento 4

Descripción

Para la implementación del requerimiento se realizó un recorrido secuencial completo sobre la estructura **array_list** que almacena los computadores en el catálogo.

Durante el recorrido se aplicaron dos filtros normalizados (marca del CPU y modelo de GPU), utilizando **strip()** y **lower()** para evitar inconsistencias por diferencias en mayúsculas, minúsculas o espacios adicionales.

Para los computadores que cumplen ambas condiciones, se calcularon en una sola pasada:

- Total de computadores que cumplen el filtro.
- Suma acumulada de precio, RAM, VRAM y frecuencia boost del CPU para el cálculo de promedios.
- Identificación de los dos computadores más costosos, aplicando criterio de desempate por menor peso.

Para determinar los dos computadores más costosos se utilizó una función auxiliar *es_mejor_candidato*, la cual compara dos computadores según:

1. Mayor precio.
2. En caso de empate, menor peso.

Este enfoque evita realizar ordenamientos adicionales sobre la lista o ciclos anidados, manteniendo la eficiencia del algoritmo.

Entrada	catalog: estructura que contiene la lista de computadores almacenada como arraylist. brand: nombre de la marca a consultar (string). cpu_brand: marca del procesador a filtrar. gpu_model: modelo de tarjeta gráfica a filtrar.
Salidas	El algoritmo retorna: · Tiempo de ejecución en milisegundos. · Número total de computadores que cumplen ambos filtros. · Promedio de: - o Precio - o Memoria VRAM - o Memoria RAM - o Frecuencia boost del CPU · Lista con los dos computadores más costosos que cumplen el filtro (con desempate por menor peso).
Implementado (Sí/No)	Si. Implementado por todo el grupo.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tamaño del catálogo	O (1)
Normalización de filtros	O (1)
Inicialización de variables acumuladoras	O (1)

Recorrido completo del catálogo	$O(n)$
Evaluación de filtros	$O(1)$
Llamada a función auxiliar <i>es_mejor_candidato</i>	$O(1)$
Cálculo de promedios	$O(1)$
Construcción de lista de resultados	$O(1)$
TOTAL	$O(n)$

Nota

La función auxiliar *es_mejor_candidato* contiene la siguiente complejidad en cada uno de sus pasos:

Pasos	Complejidad
Verificación si c2 es None	$O(1)$
Conversión de precios a float	$O(1)$
Comparación de precios	$O(1)$
Conversión de peso a float (en caso de empate)	$O(1)$
Comparación de peso	$O(1)$
Retorno del resultado	$O(1)$
TOTAL	$O(1)$

La función no contiene ciclos ni estructuras proporcionales al tamaño de la entrada, por lo que su complejidad total es **$O(1)$** .

Análisis

La operación dominante del algoritmo es el recorrido completo del catálogo de computadores para filtrar por una marca de procesador y modelo de GPU específico.

Sea n el número total de computadores en el catálogo:

El algoritmo realiza exactamente un recorrido completo de la estructura **array_list**.

- Dentro del ciclo, todas las funciones primitivas tales como comparaciones, sumas acumulativas y llamadas a la función auxiliar *son_mejor_candidato* son, de acuerdo con la teoría y la nota del punto anterior, de tiempo constante **$O(1)$** .
- La función auxiliar no contiene ciclos ni estructuras proporcionales al tamaño de la entrada, por lo que su complejidad es **$O(1)$** . Por lo tanto, cada iteración del ciclo principal mantiene la complejidad constante.

- No se emplean estructuras auxiliares proporcionales al tamaño de la entrada.
- No se realizan recorridos adicionales ni existen ciclos anidados.

Por tanto, la complejidad temporal de este requerimiento es:

$$O(n)$$

Esta complejidad es óptima, ya que es necesario inspeccionar cada computador al menos una vez para verificar si cumple con los filtros dados por el usuario.

En cuanto a la estructura de datos utilizada, se tomó la decisión de utilizar la estructura **array_list** en lugar de **single_linked_list**. Dado que el algoritmo utiliza acceso por índice en cada iteración mediante **get_element()**, el uso de una lista enlazada simple supondría un costo adicional por acceso secuencial a cada uno de los nodos, lo que podría incrementar la complejidad a **O(n²)**. En cambio, **array_list** permite acceso **O(1)** por índice, manteniendo la complejidad total en **O(n)**.

Finalmente, en las pruebas realizadas, el tiempo de ejecución crece de manera proporcional al tamaño del dataset, lo cual confirma el comportamiento lineal esperado del algoritmo.

Requerimiento 5

Descripción:

Para la implementación del requerimiento se realizó un recorrido secuencial sobre la estructura **arraylist** que almacena los computadores en el catálogo.

Durante el recorrido se filtraron únicamente aquellos computadores cuyo año de lanzamiento estuviera dentro del rango `[min_year, max_year]` y cuya resolución coincidiera con la resolución ingresada por el usuario (aplicando `strip()` para evitar inconsistencias por espacios).

En una primera pasada sobre los datos se calcularon:

- Lista de computadores que cumplen las condiciones.
- Contador total de computadores filtrados.
- Sumas acumuladas para el cálculo de promedios de:
 - Precio
 - Tamaño del display
 - GPU tier

Posteriormente, se realizó un segundo recorrido sobre la lista filtrada para determinar el mejor computador según el criterio indicado por el parámetro `filtro`:

Si el filtro es "BARATO", se selecciona el computador de menor precio (desempate por menor peso).

Si el filtro es "CARO", se selecciona el computador de mayor precio (desempate por menor peso).

Finalmente, se calcularon los promedios y se construyó el diccionario con la información del computador seleccionado. De esta manera se evita el uso de ciclos anidados, manteniendo una complejidad lineal respecto al tamaño del catálogo.

Entrada	catalog: estructura que contiene la lista de computadores almacenada como arraylist. filtro: criterio de selección ("BARATO" o "CARO") resolución: resolución específica a filtrar (string). min_year: año mínimo a filtrar (Int) max_year: año máximo a filtrar (int)
Salidas	El algoritmo retorna: • Tiempo de ejecución en milisegundos. • Tipo de filtro aplicado. • Número total de computadores que cumplen las condiciones. • Información del mejor computador según el filtro (precio, tamaño de display, GPU tier, tipo de display, año y peso). • Promedio de: ○ Precio ○ Tamaño del display ○ GPU tier
Implementado (Sí/No)	Implementado por todos

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tiempo de inicio	$O(1)$
Obtener tamaño del catálogo	$O(1)$
Creación lista auxiliar	$O(1)$
.strip() del filtro	$O(1)$
Inicialización de variables acumuladoras	$O(1)$
primer recorrido del catálogo completo	$O(N)$
segundo recorrido lista filtrada	$O(K)$
Cálculo de promedios	$O(1)$
Construcción del diccionario de respuesta	$O(1)$
Cálculo del tiempo de ejecución	$O(1)$
TOTAL	$O(N)$

Donde:

n es el número total de computadores en el catálogo.

k es el número de computadores que cumplen las condiciones.

En el peor caso, $k = n$.

Análisis

La operación dominante del algoritmo corresponde a:

1. Un recorrido completo del catálogo de tamaño n , en el cual se realizan únicamente operaciones de tiempo constante tales como comparaciones, conversiones de tipo, sumas acumuladas y el añadir al final de la lista (`add_last`), cuya complejidad es $O(1)$.
2. Un segundo recorrido sobre la lista filtrada de tamaño k para determinar el mejor computador según el criterio indicado.

Por tanto, la complejidad temporal puede expresarse como:

$$T(n)=O(n)+O(k)$$

En el peor caso, cuando todos los computadores cumplen las condiciones de ($k = n$):

$$T(n)=O(n)$$

Esta complejidad es lineal y resulta adecuada, ya que es necesario inspeccionar cada elemento del catálogo al menos una vez para verificar si cumple las condiciones de filtrado.

Complejidad espacial

Se utiliza una lista auxiliar para almacenar los computadores que cumplen las condiciones de año y resolución.

En el peor caso, todos los elementos del catálogo podrían cumplir dichas condiciones, por lo que el espacio adicional requerido es n .

Por tanto, la complejidad espacial es:

$$O(n)$$

Justificación de la estructura de datos

Se utilizó la estructura `arraylist` debido a:

- Acceso por índice en tiempo constante $O(1)$.

- Inserción al final en tiempo constante $O(1)$ amortizado.
- Mejor rendimiento práctico por almacenamiento contiguo en memoria.

Aunque una `single_linked_list` también permitiría recorridos en $O(n)$, el acceso por índice no sería constante, lo que podría degradar el rendimiento en este requerimiento.

Conclusión

El algoritmo presenta una complejidad temporal lineal $O(n)$ en el peor caso y una complejidad espacial $O(n)$ debido al uso de una lista auxiliar.

El tiempo de ejecución crece de manera proporcional al tamaño del dataset, confirmando el comportamiento lineal esperado del algoritmo.

Requerimiento 6

Descripción

Objetivo: Analizar sistemas operativos más usados y de mayor recaudación en un rango de años.

Procedimiento:

1. Valida y ajusta el rango de años (swap si inicio > fin)
2. Recorre secuencialmente la lista de computadores (Array List)
3. Filtra por año de lanzamiento dentro del rango
4. Para cada coincidencia, agrega el computador a una Single Linked List
5. Recorre la lista filtrada y para cada computador:
 - Agrupa por sistema operativo (OS)
 - Por cada OS acumula: count, recaudo total (suma de precios), suma de pesos
 - Rastrea el computador más costoso y más barato por OS (desempate por menor peso)
6. Calcula por cada OS: promedio de precio y promedio de peso
7. Determina:
 - OS más usado (mayor count)
 - OS que más recauda (mayor recaudo total)

8. Retorna estadísticas detalladas por OS y tiempos de ejecución

Entrada	catalog: estructura que contiene la lista de computadores almacenada como arraylist. inicio: número entero que indica el límite inferior del filtro de computadores con base en el año. fin: número entero que indica el límite superior del filtro de computadores con base en el año.
Salidas	El algoritmo retorna: - Tiempo de ejecución en milisegundos. - Total de computadores que cumplieron el filtro de años. - Nombre del sistema operativo más usado, su total de registros y su total de recaudo. - Nombre del sistema operativo que más recauda, su total de registros y su total de recaudo. Por cada sistema operativo que cumple el filtro de años: - Precio promedio - Peso promedio - Computador más costoso (modelo, marca, año, CPU, GPU, precio) - Computador más barato (modelo, marca, año, CPU, GPU, precio)
Implementado (Sí/No)	Implementado por todos

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener tiempo de inicio	$O(1)$
Obtener tamaño del catálogo	$O(1)$
Inicializar contadores por OS (ventas, recaudo, listas)	$O(1)$
Recorrido completo del catálogo (filtro por año)	$O(N)$
Acumulación de datos por OS (precio, peso, count, recaudo)	$O(1)$
Actualizar computador más costoso/barato por OS	$O(1)$
Identificar OS más usado (mayor count)	$O(M)$
Identificar OS que más recauda (mayor recaudo)	$O(M)$
Cálculo de promedios por OS	$O(M)$
Construcción del diccionario de respuesta	$O(M)$
Cálculo del tiempo de ejecución	$O(1)$
TOTAL	$O(N)$

Análisis

La operación que implica la mayor complejidad temporal en el requerimiento 6, es el recorrido completo del catálogo y computadores para filtrar por el rango de años y agrupar la información por sistema operativo.

Sea n el número total de computadores en el catálogo, m la cantidad de computadores que cumplen con el filtro de años (donde $m \leq n$), y u la cantidad de sistemas operativos únicos encontrados (donde $u \leq m$):

- El algoritmo realiza un primer recorrido completo de la estructura principal `array_list` (tamaño n) para verificar el año de lanzamiento de cada elemento.
- Los elementos que cumplen la condición se almacenan en una estructura auxiliar (`single_linked_list`). Al insertar siempre al final de la lista, esta operación toma un tiempo constante $O(1)$ por cada elemento filtrado.
- Posteriormente, se realiza un segundo recorrido esta vez solo sobre la `single_linked_list` (tamaño m) para agrupar los datos mediante un diccionario nativo. Dentro de este ciclo, las operaciones de búsqueda, inserción, actualización de totales y validación de máximos/mínimos en el diccionario toman un tiempo constante $O(1)$.
- Finalmente, se realiza un tercer recorrido corto sobre las claves del diccionario donde cada llave principal es un sistema operativo (tamaño u) para calcular los promedios finales e identificar el sistema operativo más usado y el que más recauda.
- No existen ciclos anidados

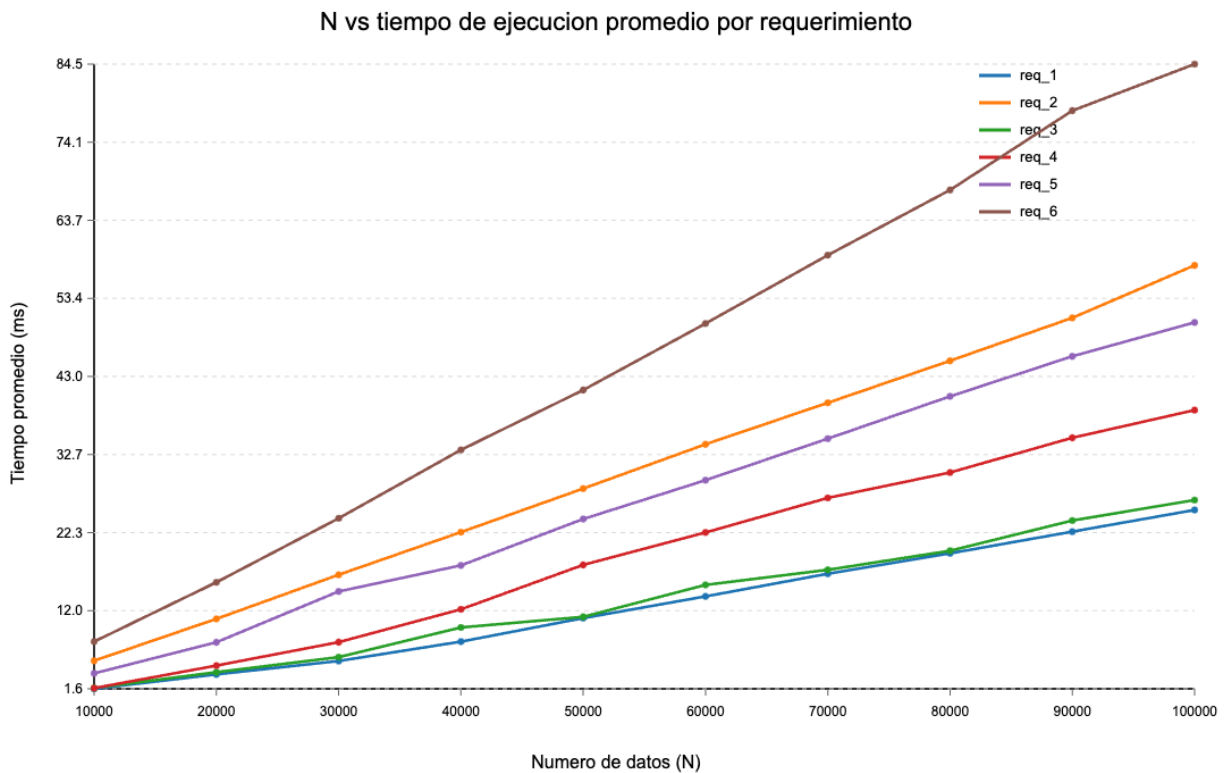
Por lo tanto, se puede concluir que la complejidad temporal del requerimiento es la suma de los recorridos que se realizan: $O(n) + O(m) + O(u)$. Es probable que el usuario seleccione un rango de años que abarca todo el dataset donde $n=m$, sin embargo la notación Big-O omite constantes multiplicativas de modo que la complejidad temporal final es $O(n)$. Experimentalmente pudimos corroborar esta conclusión como se observa en el gráfico 1.

La TAD usada para el desarrollo del requerimiento fue una single linked list para almacenar los resultados del filtro temporal. Aquí el tamaño del resultado es desconocido a priori. La lista sencillamente enlazada resulta una buena opción para este requerimiento, ya que permite un crecimiento dinámico y eficiente mediante múltiples inserciones secuenciales sin la necesidad de redimensionar bloques contiguos de memoria. Adicionalmente, el uso de diccionarios para la fase de agrupación estadística fue una decisión óptima, pues esta agrupación no tiene un gran crecimiento (son pocos sistemas operativos) y permite mantener contadores y categorizar la información por llaves (el sistema operativo) garantizando un acceso en tiempo $O(1)$.

Número de datos N vs tiempo de ejecución

Figura 1

Tiempo de ejecución promedio en función del tamaño de entrada (N) por requerimiento



Nota. El gráfico representa la complejidad temporal en función del número de datos (N) para los algoritmos asociados a cada requerimiento. El script de pruebas fue generado mediante un modelo de inteligencia artificial para asistencia en programación.

Como se observa en la **Figura 1**, se empleó un modelo de inteligencia artificial orientado a la generación de código para desarrollar un script de pruebas. Dicho script ejecuta los requerimientos req_1 a req_6 diez veces cada uno, con el fin de calcular el tiempo de ejecución promedio para distintos tamaños de entrada, específicamente:

- 10%, 20%, ..., 100% del dataset.
- Usa N real por porcentaje ($N = \text{int}(\text{total} * \text{porcentaje} / 100)$).
- Calcula por requerimiento y tamaño: avg, min, max en ms.
- Exporta resultados tabulares a CSV.
- Genera gráfica N vs tiempo promedio por requerimiento.

Estos resultados experimentales nos permiten observar el comportamiento lineal de todos los algoritmos dentro del reto. Como se explicó en las descripciones y análisis de cada requerimiento, no se usan bucles anidados, solamente uno para recorrer toda la lista en busca de los computadores que cumplan con los parámetros deseados. En ocasiones, los ítems a filtrar abarcan todo el dataset o gran parte de él, que se evidencia con una pendiente más pronunciada. Para el requerimiento 6 por ejemplo, se observa este comportamiento debido a su complejidad temporal $O(2N)$ cuando el filtro de años es 2018-2025 (todos los datos caen acá), entonces recorre toda la lista dos veces.